

Deep Learning Final Project Report

Alon Malka Amit Arad

February 2026

Abstract

Music Source Separation (MSS) is the task of isolating an individual sound source from a mixed audio signal. This project implements and compares neural network architectures for spectrogram-based vocal separation: U-Net codecs and LSTM-based models. All operated on the log-magnitude spectrograms of the data and trained via curriculum learning ($2 \rightarrow 1$ into $4 \rightarrow 3$ source separation). Our results show that U-Net models outperform the LSTM baseline across all separation quality metrics (SDR, SIR, SAR). Additionally, we experimented with adding attention mechanisms to the Codec.

1 Introduction

Vocal separation is a sub-task of Music Source Separation (MSS), which means decomposing a mixed audio signal into its separate tracks (vocals, guitars, piano, etc.) It is still being researched and is our main objective in the final project. Our preliminary work was to examine recent works and research in the field. We concluded that many approaches benefit from using U-Nets or LSTMs based models to solve this problem. The idea focuses on modeling image-to-image mappings, where the input image is the spectrogram of the song and the output is a predicted mask, to be multiplied by the input mixture in order to mask out the vocals. In our project, we decided to focus on the U-Net architecture, implementing a standard U-Net and comparing it with the LSTM model. Additionally, we tried equipping the U-Net architecture with a multi-head attention block between the encoder and the decoder parts. Training was performed on a data set named "MUSDB18", a standard in the field. It contains approximately 10 hours of songs and their 4 different stems.

2 Related Work

The work of Derby et al. [1] motivated working in a mask-predicting approach. In their paper, they tackle 2 main problems, the one we are concerned with is MSS. We adapted their LSTM-based model as a baseline architecture, and tried to outperform it (we didn't directly use their model, but implemented

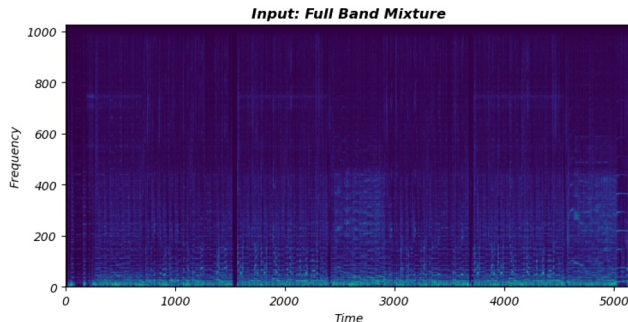


Figure 1: Spectrogram of 'Angels in Amplifiers' from the dataset

an architecture that follows the same approach using *torch*). Moreover, we discovered that our initial idea of predicting a mask (and not the spectrogram itself) was also used by Derby et al. [1].

2.1 Our Focus

We chose to tackle the complementary problem of the one Derby et al. [1] were addressing, isolating the accompaniment from the vocals. The first part of the runnable notebook features a comparison between a bidirectional LSTM-based model and our U-Net architecture. The second part of the notebook examines an improvement to the U-Net model, adding a multi-head attention block to the U-Net architecture. For the models to be able to run inference on varying duration songs, we trained them on 8 second chunks of the songs, with 4 seconds overlap. We experimented with curriculum learning, i.e., trained all of the models in two separate stages, which are explained in the training section of the methodology.

3 Methodology

In this section, we will converse some mathematical background behind signal processing, that led us to the specific architectures we implemented, the main pipeline and flow of the project. Naturally, we start with the data and the way we processed it, then we move to the models and training.

3.1 The Data

Background In digital signal processing (DSP), analyzing the raw time-domain signal $x[n]$ is usually inefficient. This is why the Discrete Fourier Transform (DFT) is being used:

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-i2\pi k \frac{n}{N}}$$

MUSDB18

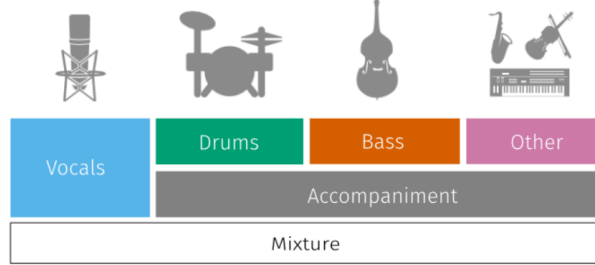


Figure 2: MUSDB18 tracks

While the DFT reveals the frequency composition of a signal it lacks temporal resolution, since it averages frequency across the time domain. To capture the frequency composition as a function of time, we use the Short-Time Fourier Transform (STFT). By using a sliding window function $w[n]$, we calculate the Fourier transform for small segments of the signal:

$$X[m, k] = \sum_{n=0}^{N-1} x[n + m \cdot H] \cdot w[n] e^{-i2\pi k \frac{n}{N}} \in \mathbb{C}$$

The magnitude of $X[m, k]$ is called the spectrogram (an illustration can be seen in Fig. 1). It is common to omit the phase (the angle of the complex number) and use only its magnitude during analysis since the phase prediction is a more complicated task, due to its cyclic nature. The phase is kept from the original signal and is only used for reconstruction of the audio signal. By converting audio into a spectrogram we essentially reduce the task to processing 2D images. This allows us to leverage computer vision architectures taught in class, like CNNs and U-Nets for our task.

The Data Set We used a database called MUSDB18 which is a standard in the field. It has approximately 10 hours of songs (signals). The signals are stereophonic and encoded at 44.1 kHz (songs on most popular music streaming are encoded at this rate). Each song has five tracks (stems): 'vocals', 'drums', 'bass', 'other' and 'mixture' as can be seen in Fig. 2. As input, we take a linear combination of these 4 stems (the initial idea, which was later abandoned, was to assign changing weight ratios as the coefficients of this linear combination in order for the model to be more robust when facing various types of songs), and as for the output, we predict a mask of the same input size (with values ranging from 0 to 1), that is to be multiplied by the input spectrogram elementwise to isolate the vocals.

Data processing Working with audio files can be challenging due to their size. For a simple 3 minutes song sampled at 44.1kHz, one must allocate memory space proportional to $44100 \cdot 180 \approx 8M$ which is huge. To overcome this, we downsample the audio by a factor of 2, converting it to 22.05kHz. We then split the musdb18 songs into chunks of 8 seconds with 4 seconds overlap, i.e., some 16 seconds of a song will be divided into three batches, 0-8, 4-12, 8-16. Then, the STFT of a chunk is being calculated, with $n_fft = 2048$ and $hop_length = 512$, resulting in a 1025×341 spectrograms that serve as input to the model.

3.2 Models

All models are trained to predict a mask of the same size as the input spectrogram, with values ranging from 0 to 1, that is later multiplied by the input spectrogram elementwise to isolate the vocals. The .npy files (chunks) turn into spectrograms using STFT and then their log magnitude is calculated. The loss function we used is the mean squared error (MSE) between the log magnitude of the predicted spectrogram and the target spectrogram ($loss = MSE(est_log, tgt_log)$). Then, for reconstructing the 1-D audio signal back, we use the Inverse Short Time Fourier Transform (ISTFT) with the phase of the original signal.

U-Net Our main model in this project was the U-Net. As mentioned, the idea is that via signal processing tools, we can convert the task to image-to-image translation, hence it was natural to use a convolutional based architecture. Essentially, the U-Net follows an Encoder-Decoder (Codec) architecture. The encoder consists of down-sample convolutional blocks reducing spatial information. The decoder does the opposite with up-sample blocks. In addition, between each layer we added concatenated skip-connections that connect corresponding layers in the encoder and decoder, as can be seen in Fig. 3, to allow preservation of spatial information and fine grained details. The classes which we implemented are: *ConvLayer2D*, *EncoderBlock*, *DecoderBlock* and *TimeFrequencyDomainUNet*.

LSTM The baseline architecture was inspired from the work of Derby et al. [1]. It consists of a Two-Layer bidirectional LSTM-based model, with the spectrograms as a sequential input. Each spectrogram (chunk that went through the STFT) is viewed as a frequencies-vector sequence. The resulting sequence is fed to an embedding layer (two fully connected layers, with ReLU activation, dropout and sigmoid) that predicts the mask. The class we implemented was *SpectrogramMaskingLSTM* that encapsulates the torch LSTM module.

Unettention (U-Net + Attention) The next part of the notebook displays experiments with the capabilities of the attention mechanism to improve the U-Net model. . The classes we implemented are *MultiHeadSelfAttention2D* and *UNetAttention*.

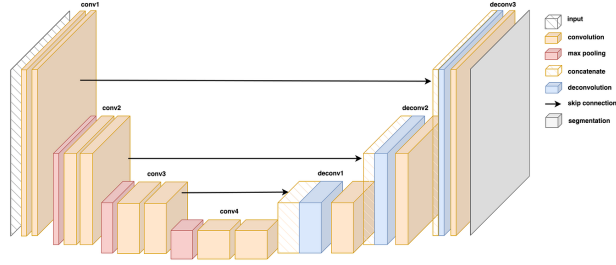


Figure 3: The U-Net architecture illustrated. The encoder and decoder are connected via skip connections.

3.3 Training

We experimented with curriculum learning, i.e., trained the model in two separate stages. The first was $2 \rightarrow 1$ source separation. The vocals + one other random track were merged and given as input. After 50 epochs, stage two was initialized with the task of $4 \rightarrow 1$ source separation. The model continued training with the weights it had from stage one.

4 Experiments

5 Work Report

Bla Bla

The evaluation metrics we used are the standard in the field: Signal to Distortion Ratio (SDR), Signal to Interference Ratio (SIR) and Signal to Artifacts Ratio (SAR).

References

- [1] B. Derby, L. Dunker, S. Galchar, S. Jarmale, and A. Setti, “Source Separation & Automatic Transcription for Music,” Submitted on 9 Dec 2024.